

Практическая теория типов

Лекция 7: Соответствие Карри-Ховарда

Воронов Михаил Сергеевич

ВМК МГУ

Осень 2025

План лекции

- 1 Введение в формальные логики
- 2 Натуральный вывод
- 3 Изоморфизм доказательств и программ
- 4 Детуры, нормализация и консистентность
- 5 Классическая логика (НК)
- 6 Модели интуиционистской логики и нормализация
 - Семантика Крипке для интуиционистской логики
 - Нормализация через логические предикаты
- 7 Заключение

План лекции

- 1 Введение в формальные логики
- 2 Натуральный вывод
- 3 Изоморфизм доказательств и программ
- 4 Детуры, нормализация и консистентность
- 5 Классическая логика (NK)
- 6 Модели интуиционистской логики и нормализация
 - Семантика Крипке для интуиционистской логики
 - Нормализация через логические предикаты
- 7 Заключение

Что такое формальная логика?

- Формальная система, состоящая из языка, набора аксиом и правил вывода
- Основная задача — изучение справедливости утверждений и способа их выводимости
- Используется часто для построения систем типов языков программирования

Свойства формальных логик

- Консистентность — $\nexists A : \Gamma \vdash A$ и $\Gamma \vdash \neg A$, эквивалентно: $\Gamma \not\vdash \perp$
- Разрешимость — для пропозициональной логики существует алгоритм, решающий выводимость формул. Для логики первого порядка общая задача выводимости неразрешима (теорема Чёрча)
- Soundness — если выражение $\Gamma \vdash A$ выводимо, то $\Gamma \models A$
- Completeness — если выражение $\Gamma \models A$, то $\Gamma \vdash A$

Обозначения: $\Gamma \vdash A$ означает « A выводимо из Γ синтаксически»,

$\Gamma \models A$ означает « A истинно при любой интерпретации, где истинны все формулы из Γ »

Единой универсальной логики не существует, разные логики моделируют разные аспекты рассуждений:

- Какие свойства мы хотим от доказательств (конструктивность, ресурсы, время)
- Что мы считаем истинным (закон исключённого третьего, противоречия)
- Какие вычисления моделируем (детерминизм, параллелизм, побочные эффекты)

Примеры формальных логик:

- Интуиционистская
- Классическая
- Линейная
- Аффинная
- Упорядоченная
- Релевантная
- Логика первого порядка (исчисление предикатов)
- Логики высоких порядков

Пропозициональная логика

Что такое пропозициональная логика?

Это простейшая формальная логика, работающая с высказываниями (утверждениями, которые могут быть истинными или ложными).

Зачем она нужна?

- Основа для более сложных логик — фундамент для логики предикатов, модальной логики и др.
- Верификация программ и схем — проверка корректности алгоритмов и электронных схем
- SAT-солверы — решение задач выполнимости (планирование, оптимизация, ИИ)
- Системы типов — основа для типизированных языков программирования (соответствие Карри-Ховарда)
- Формализация рассуждений — строгое доказательство математических утверждений

Пропозициональная логика

Пропозициональная (высказывательная) логика:

- Атомарные формулы: $P, Q, R, \dots$ — элементарные высказывания
- Логические связки:
 - $A \wedge B$ — конъюнкция (и)
 - $A \vee B$ — дизъюнкция (или)
 - $A \Rightarrow B$ — импликация (если..., то...)
 - $\neg A$ — отрицание (не)
 - $\top$ — истина (всегда верно)
 - $\perp$ — ложь (всегда неверно)
- Синтаксис — правила построения формул
- Семантика — значение формул (истина/ложь)

Замечание: В классической логике используются таблицы истинности, в интуиционистской — конструктивные доказательства.

От неформальных рассуждений к формальным

Пример неформального рассуждения:

«Если идет дождь, то земля мокрая. Идет дождь. Следовательно, земля мокрая.»

Формализация:

- Обозначим: P = «идет дождь», Q = «земля мокрая»
- Посылки: $P \Rightarrow Q$ и P
- Заключение: Q
- Правило вывода: *modus ponens*

Зачем нужна формализация?

- 1 Точность — исключает неоднозначность естественного языка
- 2 Проверимость — можно механически проверить корректность
- 3 Автоматизация — компьютер может проверять доказательства автоматически
- 4 Универсальность — одинаково работает в любой области

Формализация доказательств

Рассмотрим формулу (непрерывность функции $f(x) = 2x$ в точке 0):

$$\forall \varepsilon. (\varepsilon > 0 \Rightarrow \exists \eta. (\eta > 0 \wedge \forall x. |x| < \eta \Rightarrow |2x| < \varepsilon))$$

Примечание: это пример интуитивной формализации с кванторами $\forall, \exists$ — их семантика будет рассмотрена в расширениях NJ (логика первого порядка)

Идея доказательства:

- ❶ Выбираем $\eta = \varepsilon/2$
- ❷ Доказываем, что $\eta > 0$ (следует из $\varepsilon > 0$)
- ❸ Доказываем, что $|x| < \varepsilon/2 \Rightarrow |2x| < \varepsilon$
- ❹ Применяем $\exists$ -введение и $\Rightarrow$ -введение
- ❺ Применяем $\forall$ -введение

Дерево вывода можно построить используя правила NJ, комбинируя:

- Арифметические факты ($\varepsilon/2 \times 2 = \varepsilon$)
- Логические правила ($\forall$ -intro, $\exists$ -intro, $\Rightarrow$ -intro, $\wedge$ -intro)

План лекции

- 1 Введение в формальные логики
- 2 **Натуральный вывод**
- 3 Изоморфизм доказательств и программ
- 4 Детуры, нормализация и консистентность
- 5 Классическая логика (NK)
- 6 Модели интуиционистской логики и нормализация
 - Семантика Крипке для интуиционистской логики
 - Нормализация через логические предикаты
- 7 Заключение

Системы доказательств: обзор

Как формализовать логические рассуждения?

Существует несколько подходов к построению формальных систем доказательств:

- ❶ Гильбертовские исчисления (аксиоматический подход)
 - Много аксиом, мало правил вывода (часто только *modus ponens*)
 - Компактны, но доказательства громоздки и неинтуитивны
 - Пример: система Гильберта для пропозициональной логики
- ❷ Натуральный вывод (без аксиом-схем)
 - Нет аксиом-схем, есть правило (*hup*) для использования предположений
 - Много правил вывода (для каждой связки)
 - Доказательства близки к естественным рассуждениям
 - Пример: системы НК (классическая) и NJ (интуиционистская)
- ❸ Секвенциальное исчисление
 - Симметричные правила для левой и правой частей секвенции
 - Удобно для автоматического поиска доказательств
 - Пример: исчисление Генцена LK и LJ

Натуральный вывод: история и мотивация

- Формализм для доказательств, разработанный Герхардом Генценом и Станиславом Яськовским в 1934-1935
- Мотивация: сделать формальные доказательства похожими на рассуждения математиков
- Основан на подходе без аксиом-схем:
 - Вместо аксиом-схем (как у Гильберта) — структурные правила вывода
 - Есть правило (hur) для использования предположений
 - Для каждой логической связки: правило введения и правило удаления
 - Правила отражают смысл связок напрямую
- Наиболее известные системы натурального вывода:
 - NK (Natural deduction for Klassical logic) — для классической логики
 - NJ (Natural deduction for Intuitionistic logic) — для интуиционистской логики
- Ключевое свойство: нормализация доказательств (устранимость детуров)

Сравнение: аксиоматика vs. натуральный вывод

Задача: доказать $(A \Rightarrow B) \Rightarrow ((B \Rightarrow C) \Rightarrow (A \Rightarrow C))$ (транзитивность)

Гильбертовское исчисление:

Аксиомы:

- ❶ $A \Rightarrow (B \Rightarrow A)$
- ❷ $(A \Rightarrow (B \Rightarrow C)) \Rightarrow ((A \Rightarrow B) \Rightarrow (A \Rightarrow C))$
- ❸ ...

Правило: Modus Ponens

Доказательство: громоздкая цепочка применений аксиом 1, 2 и МР (6-8 шагов)

Натуральный вывод:

Аксиом: нет

Правила: $\Rightarrow$ -intro, $\Rightarrow$ -elim

Доказательство:

$$\begin{array}{c} \frac{B \Rightarrow C \quad \frac{A \Rightarrow B \quad A}{B} \Rightarrow E}{C} \Rightarrow E \\ \hline A \Rightarrow C \Rightarrow I \\ \hline (B \Rightarrow C) \Rightarrow (A \Rightarrow C) \Rightarrow I \\ \hline (A \Rightarrow B) \Rightarrow ((B \Rightarrow C) \Rightarrow (A \Rightarrow C)) \Rightarrow I \end{array}$$

Почему натуральный вывод?

Преимущества безаксиоматического подхода:

- ❶ Интуитивность — правила соответствуют естественным рассуждениям
 - «Если у меня есть A и B , то у меня есть $A \wedge B$ »
 - «Если из предположения A я вывожу B , то верно $A \Rightarrow B$ »
- ❷ Модульность — для каждой связки есть правила введения и удаления
 - Легко добавлять новые связки
 - Легко изучать поведение отдельных связок
- ❸ Соответствие Карри-Ховарда — рассмотрим дальше
- ❹ Нормализация — доказательства можно упрощать (устранимость детуров)

NJ: формализм

Формулы — строятся из атомарных высказываний (X) и логических связок:

$$A, B ::= X \mid A \Rightarrow B \mid A \wedge B \mid \top \mid A \vee B \mid \perp \mid \neg A$$

Примеры: $P \Rightarrow Q$, $(P \wedge Q) \Rightarrow R$, $\neg(P \vee Q)$

Контекст Γ — набор предположений (гипотез):

$$\Gamma = A_1, \dots, A_n$$

Интуиция: «что мы уже знаем», «какие факты у нас есть»

Суждение (секвент) — утверждение о выводимости:

$$\Gamma \vdash A$$

Читается: «из предположений Γ можно вывести A »

Примеры: $P, P \Rightarrow Q \vdash Q$ (modus ponens), $\vdash P \Rightarrow P$ (рефлексивность)

NJ: правила вывода

Базовое правило (гипотеза/идентичность):

$$\frac{}{\Gamma, A, \Gamma' \vdash A} \text{ (hyp)}$$

Интуиция: если A есть среди наших предположений, мы можем использовать A

Примечание: в NJ структурные правила (ослабление, перестановка, сжатие) допустимы, а не примитивны

Импликация:

$$\frac{\Gamma \vdash A \Rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} (\Rightarrow_E)$$

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \Rightarrow B} (\Rightarrow_I)$$

Конъюнкция:

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (\wedge^L_E) \quad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} (\wedge^R_E)$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (\wedge_I)$$

NJ: правила вывода (продолжение)

Истина:

$$\frac{}{\Gamma \vdash \top} (\top_I)$$

Дизъюнкция:

$$\frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C} (\vee_E)$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} (\vee_I^L) \quad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} (\vee_I^R)$$

Замечание: $\neg A$ определяется как $A \Rightarrow \perp$

Ложь:

$$\frac{\Gamma \vdash \perp}{\Gamma \vdash A} (\perp_E)$$

Отрицание:

$$\frac{\Gamma \vdash \neg A \quad \Gamma \vdash A}{\Gamma \vdash \perp} (\neg_E)$$

$$\frac{\Gamma, A \vdash \perp}{\Gamma \vdash \neg A} (\neg_I)$$

Цветовая схема правил

Обозначения в лекции

Для улучшения восприятия правила вывода выделены цветом:

- Синий цвет — правила введения (Introduction): строят новую связку
- Красный цвет — правила удаления (Elimination): используют связку
- Зелёный цвет — базовое правило (hyp) для гипотез

Аксиомы vs. гипотезы

В NJ нет аксиом-схем Правило (hyp) — базовое правило для гипотез (предположений). Аксиома-схема — универсальная истина, верная всегда (пример: $A \Rightarrow (B \Rightarrow A)$ в Гильберте).

Гипотеза — локальное допущение в контексте Γ , которое можно добавлять ($\Rightarrow_I$) и убирать.

NJ — без аксиом-схем: мы не можем использовать «готовые истины», только то, что явно предположили.

Интуиция за правилами вывода

Импликация:

($\Rightarrow_I$): «Чтобы доказать $A \Rightarrow B$, предположи A и докажи B »

($\Rightarrow_E$): «Если есть $A \Rightarrow B$ и есть A , получаем B »

(это modus ponens)

Конъюнкция:

($\wedge_I$): «Чтобы доказать $A \wedge B$, докажи и A , и B »

($\wedge^L_E$), ($\wedge^R_E$): «Из $A \wedge B$ можно извлечь A или B »

Истина:

($\top_I$): « $\top$ всегда доказуемо без посылок»

Дизъюнкция:

($\vee^L_I$), ($\vee^R_I$): «Чтобы доказать $A \vee B$, достаточно доказать A или B »

($\vee_E$): «Чтобы использовать $A \vee B$, нужно разобрать оба случая»

Ложь:

($\perp_E$): «Из лжи следует всё»
(ex falso quodlibet)

Если доказано $\perp$, значит наши предположения противоречивы, а из них можно доказать что угодно

Отрицание:

($\neg_I$): «Чтобы доказать $\neg A$, предположи A и выведи противоречие»

($\neg_E$): «Из $\neg A$ и A получаем противоречие $\perp$ »

Пример 1: $A \Rightarrow A$ (рефлексивность)

$$\frac{\overline{A \vdash A} \text{ (hyp)}}{\vdash A \Rightarrow A} (\Rightarrow_I)$$

Пример 2: $A \Rightarrow (B \Rightarrow A)$
(weakening)

$$\frac{\frac{\overline{A, B \vdash A} \text{ (hyp)}}{A \vdash B \Rightarrow A} (\Rightarrow_I)}{\vdash A \Rightarrow (B \Rightarrow A)} (\Rightarrow_I)$$

Примеры вывода

Пример 3: $(A \Rightarrow (B \Rightarrow C)) \Rightarrow ((A \Rightarrow B) \Rightarrow (A \Rightarrow C))$

$$\frac{\frac{\frac{\overline{\Gamma \vdash A \Rightarrow (B \Rightarrow C)}}{\Gamma \vdash B \Rightarrow C} \text{ (hyp)} \quad \frac{\overline{\Gamma \vdash A}}{\Gamma \vdash A} \text{ (hyp)}}{\Gamma \vdash B \Rightarrow C} \text{ (}\Rightarrow\text{E)} \quad \frac{\frac{\overline{\Gamma \vdash A \Rightarrow B}}{\Gamma \vdash B} \text{ (hyp)} \quad \frac{\overline{\Gamma \vdash A}}{\Gamma \vdash A} \text{ (hyp)}}{\Gamma \vdash B} \text{ (}\Rightarrow\text{E)} \quad \frac{\Gamma \vdash B \quad \Gamma \vdash B \Rightarrow C}{\Gamma \vdash C} \text{ (}\Rightarrow\text{E)} \quad \frac{A \Rightarrow (B \Rightarrow C), A \Rightarrow B \vdash A \Rightarrow C}{A \Rightarrow (B \Rightarrow C) \vdash (A \Rightarrow B) \Rightarrow (A \Rightarrow C)} \text{ (}\Rightarrow\text{I)} \quad \frac{A \Rightarrow (B \Rightarrow C) \vdash (A \Rightarrow B) \Rightarrow (A \Rightarrow C)}{\vdash (A \Rightarrow (B \Rightarrow C)) \Rightarrow ((A \Rightarrow B) \Rightarrow (A \Rightarrow C))} \text{ (}\Rightarrow\text{I)}$$

где $\Gamma = A \Rightarrow (B \Rightarrow C), A \Rightarrow B, A$

Первое применение, второе применение, промежуточные результаты, финальный результат

Пример: Закон противоречия

Доказательство: $\neg(A \wedge \neg A)$

$$\frac{\frac{\overline{A \wedge \neg A \vdash A \wedge \neg A} \text{ (hyp)}}{A \wedge \neg A \vdash \neg A} (\wedge^R_E) \quad \frac{\frac{\overline{A \wedge \neg A \vdash A \wedge \neg A} \text{ (hyp)}}{A \wedge \neg A \vdash A} (\wedge^L_E)}{A \wedge \neg A \vdash \perp} (\neg E)$$

$$\vdash \neg(A \wedge \neg A) \quad (\neg I)$$

Примеры вывода

Доказательство: $(A \Rightarrow B) \Rightarrow (\neg B \Rightarrow \neg A)$

$$\begin{array}{c} \frac{}{A \Rightarrow B, \neg B, A \vdash \neg B} \text{ (hyp)} \\ \frac{}{A \Rightarrow B, \neg B, A \vdash A \Rightarrow B} \text{ (hyp)} \quad \frac{}{A \Rightarrow B, \neg B, A \vdash A} \text{ (hyp)} \\ \hline \frac{}{A \Rightarrow B, \neg B, A \vdash B} \text{ (}\Rightarrow\text{E)} \\ \hline \frac{}{A \Rightarrow B, \neg B, A \vdash \perp} \text{ (}\neg\text{E)} \\ \hline \frac{}{A \Rightarrow B, \neg B \vdash \neg A} \text{ (}\neg\text{I)} \\ \hline \frac{}{A \Rightarrow B \vdash \neg B \Rightarrow \neg A} \text{ (}\Rightarrow\text{I)} \\ \hline \frac{}{\vdash (A \Rightarrow B) \Rightarrow (\neg B \Rightarrow \neg A)} \text{ (}\Rightarrow\text{I)} \end{array}$$

Упражнение: каррирование

Задание

Докажите: $(A \Rightarrow (B \Rightarrow C)) \Rightarrow ((A \wedge B) \Rightarrow C)$

Наблюдение:

- Это называется каррирование (экспоненциальный изоморфизм) в программировании
- Функция двух аргументов $\equiv$ функция, возвращающая функцию

Попробуйте также доказать обратное направление (декаррирование):

$$((A \wedge B) \Rightarrow C) \Rightarrow (A \Rightarrow (B \Rightarrow C))$$

Факт: эти два направления вместе показывают изоморфизм

$$A \rightarrow (B \rightarrow C) \cong (A \times B) \rightarrow C$$

План лекции

- 1 Введение в формальные логики
- 2 Натуральный вывод
- 3 Изоморфизм доказательств и программ**
- 4 Детуры, нормализация и консистентность
- 5 Классическая логика (NK)
- 6 Модели интуиционистской логики и нормализация
 - Семантика Крипке для интуиционистской логики
 - Нормализация через логические предикаты
- 7 Заключение

Изоморфизм доказательств и программ

Соответствие Карри-Ховарда

Существует глубокая связь между:

- Логическими формулами и типами
- Доказательствами и программами
- Упрощением доказательств и вычислением

Логика	Программирование
Формула A	Тип A
Доказательство A	Терм типа A
$A \Rightarrow B$	Функция $A \rightarrow B$
$A \wedge B$	Произведение $A \times B$
$A \vee B$	Сумма $A + B$
$\perp$	Пустой тип
$\top$	Единичный тип

Соответствие: правила вывода и правила типизации

Логика $(\Rightarrow)$ $\leftrightarrow$ Типы $(\rightarrow)$

Логика (NJ):

Импликация: $A \Rightarrow B$

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \Rightarrow B} (\Rightarrow_I)$$

$$\frac{\Gamma \vdash A \Rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} (\Rightarrow_E)$$

Типы (λ -исчисление):

Функция: $A \rightarrow B$

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} (\text{Abs})$$

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B} (\text{App})$$

Ключевая идея: Правила **введения** ($\Rightarrow_I$) соответствуют абстракции, правила **удаления** ($\Rightarrow_E$) — применению функции.

Соответствие символов: Логика $\Rightarrow$ (импликация) $\leftrightarrow$ Типы $\rightarrow$ (функция)

Соответствие: конъюнкция и произведение

Логика $(\wedge) \leftrightarrow$ Типы $(\times)$

Логика:

Конъюнкция: $A \wedge B$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (\wedge_I)$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (\wedge^L_E) \quad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} (\wedge^R_E)$$

Типы:

Произведение: $A \times B$

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash N : B}{\Gamma \vdash (M, N) : A \times B} (\text{Pair})$$

$$\frac{\Gamma \vdash P : A \times B}{\Gamma \vdash \pi_1(P) : A} (\text{Fst})$$

$$\frac{\Gamma \vdash P : A \times B}{\Gamma \vdash \pi_2(P) : B} (\text{Snd})$$

Соответствие символов: Логика $\wedge$ (конъюнкция) $\leftrightarrow$ Типы $\times$ (произведение/пара)

Соответствие: дизъюнкция и сумма

Логика $(\vee) \leftrightarrow$ Типы $(+)$

Логика:

Дизъюнкция: $A \vee B$

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} (\vee_I^L) \quad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} (\vee_I^R)$$

$$\frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C} (\vee_E)$$

Типы:

Сумма: $A + B$

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash \text{inl}(M) : A + B} (\text{InL})$$

$$\frac{\Gamma \vdash N : B}{\Gamma \vdash \text{inr}(N) : A + B} (\text{InR})$$

$$\frac{\Gamma \vdash P : A + B \quad \Gamma, x : A \vdash M : C \quad \Gamma, y : B \vdash N : C}{\Gamma \vdash \text{case } P \{x.M, y.N\} : C} (\text{Case})$$

Соответствие символов: Логика $\vee$ (дизъюнкция) $\leftrightarrow$ Типы $+$ (сумма/variant)

Пример: Коммутативность дизъюнкции

Логическая формула: $(A \vee B) \Rightarrow (B \vee A)$

Соответствующий тип: $(A + B) \rightarrow (B + A)$

Программа (терм):

$$\lambda p. \text{case } p \{x.\text{inr}(x), y.\text{inl}(y)\}$$

Интерпретация

- Функция принимает значение суммарного типа $A + B$
- Если это $\text{inl}(x)$ (т.е. $x : A$), возвращает $\text{inr}(x) : B + A$
- Если это $\text{inr}(y)$ (т.е. $y : B$), возвращает $\text{inl}(y) : B + A$

Пример: Закон противоречия

Логическая формула: $\neg(A \wedge \neg A)$

Эквивалентно: $(A \wedge (A \rightarrow \perp)) \rightarrow \perp$

Соответствующий тип: $(A \times (A \rightarrow \perp)) \rightarrow \perp$

Программа:

$$\lambda p. (\pi_2(p)) (\pi_1(p))$$

Интерпретация

- Функция принимает пару: значение типа A и функцию из A в $\perp$
- Применяем функцию $\pi_2(p) : A \rightarrow \perp$ к аргументу $\pi_1(p) : A$
- Получаем $\perp$ (абсурд, пустой тип)

Пример: Дистрибутивность

Формула: $A \wedge (B \vee C) \Rightarrow (A \wedge B) \vee (A \wedge C)$

Тип: $A \times (B + C) \rightarrow (A \times B) + (A \times C)$

Программа:

$$\lambda p. \text{case } (\pi_2(p)) \\ \{b.\text{inl}((\pi_1(p), b)), \\ c.\text{inr}((\pi_1(p), c))\}$$

Интерпретация

Функция распаковывает пару $(a, \text{inl}(b))$ или $(a, \text{inr}(c))$ и переупаковывает её в $\text{inl}((a, b))$ или $\text{inr}((a, c))$ соответственно.

Необитаемые типы и недоказуемые формулы

Закон исключённого третьего

Формула $A \vee \neg A$ не доказуема в интуиционистской логике.

Соответствующий тип: $A + (A \rightarrow \perp)$

Нет общего терма этого типа (без дополнительных аксиом)

Закон Пирса

Формула $((A \Rightarrow B) \Rightarrow A) \Rightarrow A$ также не доказуема интуиционистски.

Соответствующий тип: $((A \rightarrow B) \rightarrow A) \rightarrow A$

Нет замкнутого терма этого типа в просто типизированном λ -исчислении.

Вывод: Необитаемые типы соответствуют недоказуемым формулам.

- Верификация программ: доказательство корректности = программа
- Автоматические доказательства: поиск термов заданного типа
- Извлечение программ из доказательств (program extraction)
- Зависимые типы: расширение соответствия до логики высших порядков
- Proof assistants: Coq, Agda, Lean основаны на этом соответствии

Принцип

Propositions as Types, Proofs as Programs

ВНК-интерпретация: доказательства как данные

Brouwer-Heyting-Kolmogorov интерпретация — семантика интуиционистской логики:

Формула	Доказательство	Тип/Программа
$A \wedge B$	Пара (a, b) , где $a : A$, $b : B$	$(a, b) : A \times B$
$A \vee B$	Выбор: либо $a : A$, либо $b : B$ + метка	$\text{Left } a : A + B$
$A \Rightarrow B$	Конструктивная функция $f : A \rightarrow B$	$\lambda a. M : A \rightarrow B$
$\perp$	Нет доказательства	Нет термов типа Void
$\top$	Тривиальное доказательство	$() : \text{Unit}$

Количество доказательств и кардинальность типа

Для алгебры сумм и произведений $(\times, +, 0, 1)$: число различных доказательств формулы = число термов соответствующего типа (с точностью до $\beta\eta$ -эквивалентности)

Пример: $(P \wedge Q) \vee (P \wedge R)$ при $|P| = 2, |Q| = 3, |R| = 5$

Число доказательств: $(2 \times 3) + (2 \times 5) = 6 + 10 = 16$

Для функций $(\rightarrow)$ подсчёт зависит от семантики (экстенсиональная/интенсиональная) и базы констант — это лишь интуиция, требующая оговорок

Отрицание и классическая логика

Отрицание в интуиционистской логике: $\neg A \equiv A \Rightarrow \perp$

В типах: $\neg A \cong A \rightarrow \text{Void}$

Алгебраически: $\neg A = 0^A$

Почему классическая логика не согласуется с вычислимостью?

В классической логике: $\neg\neg A \Leftrightarrow A$ (закон двойного отрицания)

В алгебре типов: $\neg\neg A = (A \rightarrow \text{Void}) \rightarrow \text{Void}$

Это не изоморфно A конструктивно.

Пример: Для $A = \text{Bool}$:

- $\neg A = \text{Bool} \rightarrow \text{Void}$ — пустой тип (0 функций)
- $\neg\neg A = (\text{Bool} \rightarrow \text{Void}) \rightarrow \text{Void}$ — обитаем, но не изоморфен Bool
- В STLC есть разные термы типа $\neg\neg\text{Bool}$: $\lambda k. k \text{ true}$ и $\lambda k. k \text{ false}$ — не $\beta\eta$ -эквивалентны
- в STLC без констант деструкции Void эти термы не редуцируются друг к другу, и не существует изоморфизма $\neg\neg A \cong A$
- $\neg\neg(\cdot)$ — это continuation-модальность, а не простое стирание

План лекции

- 1 Введение в формальные логики
- 2 Натуральный вывод
- 3 Изоморфизм доказательств и программ
- 4 Детуры, нормализация и консистентность
- 5 Классическая логика (NK)
- 6 Модели интуиционистской логики и нормализация
 - Семантика Крипке для интуиционистской логики
 - Нормализация через логические предикаты
- 7 Заключение

NJ: детуры и локальные редукции

Детур (detour) — последовательность «введение + удаление» одной связки.

Интерпретация

Временный результат, который создается и сразу же используется — избыточный шаг

Конъюнкция:

$$\frac{\pi \quad \Gamma \vdash A \quad \pi' \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (\wedge_I)$$
$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (\wedge_E^L)$$

Упрощается к: $\frac{\pi}{\Gamma \vdash A}$

Импликация:

$$\frac{\pi \quad \Gamma, A \vdash B}{\Gamma \vdash A \Rightarrow B} (\Rightarrow_I)$$
$$\frac{\Gamma \vdash A \Rightarrow B \quad \pi' \quad \Gamma \vdash A}{\Gamma \vdash B} (\Rightarrow_E)$$

Упрощается к: $\frac{\pi[\pi'/A]}{\Gamma \vdash B}$

Нормализация как вычисление

Ключевая идея

Устранение детуров = процесс вычисления = β -редукция

Логика	Программирование
Detour elimination	β -reduction
Введение + Удаление	Применение λ -абстракции
Нормальное доказательство	Нормальная форма

Примеры:

- Импликация: $(\lambda x.M) N \rightarrow_{\beta} M[N/x]$
- Конъюнкция: $\pi_1((M, N)) \rightarrow_{\beta} M$
- Дизъюнкция: $\text{case}(\text{inl}(M), x.P, y.Q) \rightarrow_{\beta} P[M/x]$

Принцип

Каждая связка имеет своё правило устранения детура $\equiv$ свою β -редукцию

Пример: устранение детура

Формула: $(A \vee B) \wedge \neg A \vdash B$ (modus tollendo ponens)

С детуром:

$$\frac{\frac{\overline{\Gamma \vdash (A \vee B) \wedge \neg A} \text{ (hyp)}}{\Gamma \vdash A \vee B} \text{ (}\wedge^L\text{)}}{\Gamma \vdash B} \dots \dots \text{ (}\vee\text{E)}$$

Извлекаем $(A \vee B)$, чтобы сразу
использовать

Без детуров:

$$\frac{\overline{\Gamma \vdash A \vee B} \text{ (hyp)} \quad \dots \quad \dots}{\Gamma \vdash B} \text{ (}\vee\text{E)}$$

Используем $(A \vee B)$ напрямую

Вывод

Устранение детуров убирает избыточные шаги, как программная оптимизация

Пример композиции: устранение промежуточных лемм

Задача: вычислить $(f \circ g) 5$ где $g(z) = z + 3$, $f(y) = y \times 2$

Пошаговая β -редукция:

$$((\lambda x. f (g x)) 5) \rightarrow_{\beta} f (g 5) \rightarrow_{\beta} (\lambda y. y \times 2) (5 + 3) \rightarrow_{\beta} 8 \times 2 \rightarrow_{\beta} 16$$

В логике

Доказательство $(A \Rightarrow B) \wedge (B \Rightarrow C) \wedge A \vdash C$ (транзитивность)

Устранение детуров убирает промежуточную «лемму» B

Соответствие

Логический вывод = type-safe программа. Нормализация = вычисление.

NJ: нормализация доказательств

Теорема о нормализации

Всякое доказательство можно преобразовать в доказательство без детуров

Локальные редукции (примеры):

Импликация:

$$\frac{\frac{\pi \quad \Gamma, A \vdash B}{\Gamma \vdash A \Rightarrow B} \Rightarrow_I \quad \pi' \quad \Gamma \vdash A}{\Gamma \vdash B} \rightsquigarrow \frac{\pi[\pi'/A]}{\Gamma \vdash B}$$

Конъюнкция:

$$\frac{\frac{\pi \quad \Gamma \vdash A \quad \pi' \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge_I}{\Gamma \vdash A} \rightsquigarrow \frac{\pi}{\Gamma \vdash A}$$

NJ: нормализация доказательств (продолжение)

Локальные редукции (продолжение):

Дизъюнкция:

$$\frac{\frac{\pi}{\Gamma \vdash A \vee B} \quad \frac{\pi' \quad \Gamma, A \vdash C}{\Gamma \vdash C}}{\Gamma \vdash C} \quad \rightsquigarrow \quad \frac{\pi'[\pi/A]}{\Gamma \vdash C}$$

Принцип нормализации

Устранение детуров = упрощение доказательства

Каждая связка имеет своё правило локальной редукции

Жиранд о нормализации

«Логика без нормализации подобна машине без двигателя»

NJ: консистентность

Теорема (Консистентность NJ)

NJ консистентна: формула $\perp$ невыводима.

Доказательство.

Индукция по структуре доказательства.

Используем нормализацию: доказательство $\perp$ в нормальной форме невозможно, так как нет правила введения для $\perp$. □

Следствие

Для любой формулы не выводимы одновременно она сама и её отрицание

Нормализация — ключевое свойство

Любое доказательство можно преобразовать к нормальной форме (без детуров)

Это гарантирует вычислимость и консистентность системы

План лекции

- 1 Введение в формальные логики
- 2 Натуральный вывод
- 3 Изоморфизм доказательств и программ
- 4 Детуры, нормализация и консистентность
- 5 Классическая логика (НК)**
- 6 Модели интуиционистской логики и нормализация
 - Семантика Крипке для интуиционистской логики
 - Нормализация через логические предикаты
- 7 Заключение

НК: классические принципы

Классическую логику (НК) можно получить из NJ, добавив любую из классических схем, эквивалентных над NJ, например закон исключённого третьего $A \vee \neg A$, устранение двойного отрицания $\neg\neg A \Rightarrow A$, закон Пирса $((A \Rightarrow B) \Rightarrow A) \Rightarrow A$ и др.

Наличие таких принципов приводит к тому, что логика перестаёт быть конструктивной — мы не всегда можем указать, какая из альтернатив выполнена.

Классический пример неконструктивного рассуждения:

- Существуют иррациональные a и b такие, что a^b рационально
- Доказательство: Рассмотрим $\sqrt{2}^{\sqrt{2}}$. Если оно рационально, возьмём $a = b = \sqrt{2}$.
Иначе возьмём $a = \sqrt{2}^{\sqrt{2}}$, $b = \sqrt{2}$, тогда $a^b = (\sqrt{2}^{\sqrt{2}})^{\sqrt{2}} = \sqrt{2}^2 = 2$ рационально.
- Проблема: мы не знаем, какая пара (a, b) работает! Доказательство использует LEM без указания конкретного значения.

Тогда как NJ конструктивна и всегда можно извлечь конкретное доказательство.

НК: альтернативы закону исключённого третьего

Следующие схемы формул эквивалентны закону исключённого третьего над интуиционистской логикой (IL). Каждая из них превращает NJ в НК:

- 1 Excluded middle (закон исключённого третьего): $A \vee \neg A$
- 2 Double-negation elimination: $\neg\neg A \Rightarrow A$
- 3 Peirce's law: $((A \Rightarrow B) \Rightarrow A) \Rightarrow A$
- 4 De Morgan law: $\neg(\neg A \wedge \neg B) \Rightarrow A \vee B$
- 5 Material implication: $(A \Rightarrow B) \Rightarrow (\neg A \vee B)$
- 6 Обратная контрапозиция (менее стандартный эквивалент): $(\neg B \Rightarrow \neg A) \Rightarrow (A \Rightarrow B)$

И другие: Clavius' law, Tarski's formula, counter-example principle, дистрибутивность $\Rightarrow / \vee$, и т.д.

Эквивалентность: например, подстановка $B := A$ в схему 6 даёт LEM, обратно LEM доказывает все остальные

CPS и двойное отрицание

Отрицание: $\neg A \equiv A \rightarrow \text{Void}$. Двойное отрицание: $\neg\neg A \equiv (A \rightarrow \text{Void}) \rightarrow \text{Void}$.

Continuation-представление

Общий ответный тип R даёт: $\text{Cont } R \ A \equiv (A \rightarrow R) \rightarrow R$.

- CPS-перевод заменяет «возврат результата» на «передать в продолжение»
- $\neg\neg A$ — это $\text{Cont } \text{Void } A$; классические принципы кодируются через продолжения

Связь с логикой: двойное отрицание и CPS объясняют, почему классическая логика требует дополнительных эффектов управления (например, call/cc), в то время как чистая интуиционистская интерпретация остаётся нормализуемой.

Двойное отрицание и вычислительное содержание

Интерпретация: формула A = множество $[[A]]$ всех её доказательств

Импликация как функция: $A \rightarrow B = [[A]] \rightarrow [[B]]$, $\perp = \emptyset$

Отрицание: $\neg A = [[A]] \rightarrow \emptyset$

$$\begin{aligned} [[A]] \neq \emptyset &\Rightarrow \neg A \text{ пусто} \\ [[A]] = \emptyset &\Rightarrow \neg A \text{ не пусто} \end{aligned}$$

Двойное отрицание: $\neg\neg A = (\neg A) \rightarrow \emptyset$

$$\begin{aligned} [[A]] \neq \emptyset &\Rightarrow \neg\neg A \text{ не пусто} \\ [[A]] = \emptyset &\Rightarrow \neg\neg A \text{ пусто} \end{aligned}$$

Вывод

$\neg\neg A$ «забывает» конкретные доказательства, сохраняя только их наличие

$\neg\neg$ стирает вычислительное содержание доказательства A , но сам тип $\neg\neg A$ не обязательно proof-irrelevant в интенциональной теории типов

Proof-relevant vs proof-irrelevant типы

Proof-relevant типы

Важна структура доказательства (терма)

Примеры: $\mathbb{N}$, $A \times B$, $A + B$, индуктивные типы с данными

Proof-irrelevant типы

Важно только существование доказательства

Примеры: $\neg\neg A$, $\top$, тип `Prop` в `Coq/Lean`

Proof-relevant	Proof-irrelevant
Какое число?	Существует ли?
Какая пара (a, b) ?	Доказуемо ли?
Левая или правая ветка?	Истинно ли?
Содержит данные	Не содержит данных

План лекции

- 1 Введение в формальные логики
- 2 Натуральный вывод
- 3 Изоморфизм доказательств и программ
- 4 Детуры, нормализация и консистентность
- 5 Классическая логика (NK)
- 6 Модели интуиционистской логики и нормализация
 - Семантика Крипке для интуиционистской логики
 - Нормализация через логические предикаты
- 7 Заключение

Математическое моделирование: общий подход

Модель в математике

Модель — математическая структура, которая интерпретирует абстрактные символы, придавая им конкретный смысл.

Синтаксис vs Семантика:

- Синтаксис — формальные символы и правила
- Семантика — интерпретация в математических объектах

Зачем нужны модели?

- Доказывать истинность (во всех моделях)
- Доказывать независимость (найти модель, где формула ложна)

Пример

Аксиома параллельности истинна в евклидовой геометрии, но ложна на сфере $\Rightarrow$ независима от других аксиом Евклида.

Как доказать недоказуемость?

Схема: чтобы показать, что φ не доказуема:

- 1 Строим модель $\mathcal{M}$ и интерпретацию $\llbracket \cdot \rrbracket$
- 2 Доказываем корректность: если $\vdash \varphi$, то $\llbracket \varphi \rrbracket$ истинна в $\mathcal{M}$
- 3 Находим формулу φ , ложную в $\mathcal{M}$
- 4 Вывод: φ не доказуема (иначе была бы истинна по п. 2)

Важно

Интерпретация $\llbracket \cdot \rrbracket$ — это гомоморфизм из синтаксиса в семантику. Нам достаточно односторонней импликации доказуемо $\Rightarrow$ истинно, изоморфизм не обязателен.

Применение к интуиционистской логике

Наша задача: доказать, что $A \vee \neg A$ (LEM) не доказуем в NJ.

Два подхода к семантике интуиционистской логики:

- 1 Модели Крипке — «возможные миры» с частичной информацией
- 2 Алгебры Гейтинга — решётки с операциями $\wedge, \vee, \rightarrow$

Стратегия

Построить модель, в которой:

- Все NJ-доказуемые формулы истинны (корректность)
- LEM ложен (контрпример)

Модели Крипке: интуиция

Идея: формулы интерпретируются не как «истина/ложь», а как «знание в данный момент времени»

Классическая логика

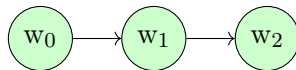
- Истинность абсолютна
- $A \vee \neg A$ всегда верно
- Одна «временная точка»



A или $\neg A$

Интуиционистская логика

- Знание монотонно растёт
- $A \vee \neg A$ может быть неизвестно
- Множество «миров» (состояний знания)



?

A

A

Ключ: в w_0 мы не знаем ни A , ни $\neg A$ — знание появляется позже!

Модели Крипке: формальное определение

Определение (Модель Крипке)

Модель Крипке для пропозициональной логики — это тройка $\mathcal{K} = (W, \leq, \Vdash)$:

- W — множество миров (worlds)
- $\leq$ — частичный порядок на W (отношение достижимости)
- $\Vdash \subseteq W \times \text{Atoms}$ — отношение forcing для атомарных формул

Монотонность атомов

Если $w \Vdash P$ и $w \leq w'$, то $w' \Vdash P$.

Интуиция: знание об атомах не теряется со временем

Обозначения:

- $w \Vdash A$ — «формула A истинна (forced) в мире w »
- $w \leq w'$ — «мир w' достижим из w » или « w' содержит больше информации, чем w »

Forcing для логических связок

Forcing $w \Vdash A$ определяется индуктивно:

- $w \Vdash P$ (атом) — задаётся явно в модели, монотонно
- $w \Vdash \top \iff$ истина (всегда)
- $w \Vdash A \wedge B \iff w \Vdash A$ и $w \Vdash B$
- $w \Vdash A \vee B \iff w \Vdash A$ или $w \Vdash B$
- $w \Vdash A \Rightarrow B \iff \forall w' \geq w. (w' \Vdash A \Rightarrow w' \Vdash B)$
- $w \Vdash \perp \iff$ ложь (никогда)
- $w \Vdash \neg A \iff \forall w' \geq w. w' \nVdash A$

Теорема о монотонности

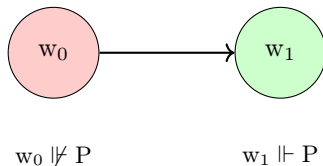
Для любой формулы A : если $w \Vdash A$ и $w \leq w'$, то $w' \Vdash A$.

Доказательство: индукция по структуре A . Импликация требует «для всех будущих миров».

Пример: контрмодель для LEM

Цель: показать, что $P \vee \neg P$ не истинна во всех моделях

Модель: два мира $w_0 \leq w_1$



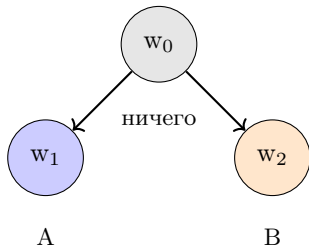
Проверка в w_0 :

- $w_0 \not\models P$ (по определению модели)
- $w_0 \not\models \neg P$, так как нужно $\forall w' \geq w_0. w' \not\models P$, но $w_1 \geq w_0$ и $w_1 \models P$
- Следовательно, $w_0 \not\models P \vee \neg P$

Вывод: существует модель Крипке, где $P \vee \neg P$ ложно $\Rightarrow$ LEM не доказуем в NJ!

Пример: контрмодель для $(A \Rightarrow B) \vee (B \Rightarrow A)$

Модель: ветвящаяся структура



Проверка в w_0 :

- $w_0 \not\models A \Rightarrow B$, так как $w_1 \geq w_0$, $w_1 \models A$, но $w_1 \not\models B$
- $w_0 \not\models B \Rightarrow A$, так как $w_2 \geq w_0$, $w_2 \models B$, но $w_2 \not\models A$
- Следовательно, $w_0 \not\models (A \Rightarrow B) \vee (B \Rightarrow A)$

Замечание: в классической логике эта формула доказуема, в NJ — нет!

Soundness и Completeness

Теорема (Soundness — корректность)

Если $\Gamma \vdash_{NJ} A$, то $\Gamma \Vdash A$ (истинно во всех моделях Крипке).

Формально: для любой модели $\mathcal{K}$ и мира w , если $w \Vdash \Gamma$, то $w \Vdash A$.

Теорема (Completeness — полнота)

Если $\Gamma \Vdash A$ (истинно во всех моделях), то $\Gamma \vdash_{NJ} A$.

Следствия

- ❶ Непротиворечивость NJ: $\perp$ недоказуемо (нет модели, где $w \Vdash \perp$)
- ❷ Независимость LEM: существует контрмодель для $A \vee \neg A$
- ❸ Метод опровержения: чтобы показать A недоказуемо в NJ, достаточно построить контрмодель Крипке

Алгебры Гейтинга: алгебраическая абстракция

Определение (Алгебра Гейтинга)

Алгебра Гейтинга — это решётка $(H, \wedge, \vee, \Rightarrow, \perp, \top)$ с операцией псевдокомплемента $\Rightarrow$:

$$a \wedge b \leq c \quad \Leftrightarrow \quad a \leq (b \Rightarrow c)$$

Примеры алгебр Гейтинга:

- ❶ Открытые множества топологии: $\mathcal{O}(X)$ для топологического пространства X
 - $U \wedge V = U \cap V$
 - $U \vee V = U \cup V$
 - $U \Rightarrow V = \text{Int}(U^c \cup V)$ (внутренность)
- ❷ Upward-closed множества в $(W, \leq)$: $\{S \subseteq W \mid w \in S, w \leq w' \Rightarrow w' \in S\}$
- ❸ Булева алгебра — частный случай, где $\neg\neg a = a$

Связь с Крипке: модель Крипке $(W, \leq, \Vdash)$ порождает алгебру Гейтинга upward-closed множеств

Мотивация: почему нужен метод Таита?

Цель: доказать сильную нормализацию (SN) для просто типизированного λ -исчисления

Теорема (которую хотим доказать)

Если $\Gamma \vdash M : \sigma$, то M сильно нормализуем (все редукции конечны).

Проблема с наивной индукцией по структуре терма:

- Редукция $(\lambda x. M) N \rightarrow_{\beta} M[x := N]$ может увеличить размер терма
- Пример: $(\lambda x. x x) I \rightarrow_{\beta} I I$ — размер вырос!
- Индукция по размеру терма не работает

Решение (метод Таита, 1967): индукция по типам, а не по термам

Идея: определить предикат «вычислимый» (reducible) через индукцию по типам

Основная идея

Доказываем сильную нормализацию косвенно через вычислимость (reducibility)

План доказательства:

- ❶ Определение: вводим предикат RED_σ для каждого типа σ
 - Индукция по структуре типа
 - Для базовых типов: $\text{RED}_\alpha = \text{SN}$
 - Для стрелок: функциональное определение
- ❷ Свойства: доказываем ключевые свойства RED_σ
 - CR_1 : вычислимые термы сильно нормализуемы
 - CR_2 : замкнутость относительно обратных редукций
 - CR_3 : замкнутость для нейтральных термов
- ❸ Fundamental Lemma: все типизуемые термы вычислимы
- ❹ Вывод: $\vdash M : \sigma \Rightarrow M \in \text{RED}_\sigma \Rightarrow M \in \text{SN}$

Метод логических предикатов (Tait)

Идея: определить множество RED_σ «вычислимых термов типа σ » так, чтобы:

- 1 Все вычислимые термы сильно нормализуемы
- 2 Все типизуемые термы вычислимы

Определение (Reducibility predicates для STLC)

Определяем RED_σ индукцией по структуре типа σ :

- Базовый тип α :

$$M \in RED_\alpha \iff M \in SN$$

- Стрелочный тип $\sigma \rightarrow \tau$:

$$M \in RED_{\sigma \rightarrow \tau} \iff \forall N \in RED_\sigma. MN \in RED_\tau$$

Интуиция: терм типа $\sigma \rightarrow \tau$ вычислим, если он «ведёт себя как функция» — применённый к вычислимым аргументам даёт вычисляемый результат

Основные свойства (CR_1 , CR_2 , CR_3)

Лемма (CR_1 — Closure under SN)

Если $M \in RED_\sigma$, то $M \in SN$.

Лемма (CR_2 — Closure under expansion)

Если $M \rightarrow_\beta M'$ и $M' \in RED_\sigma$, то $M \in RED_\sigma$.

Лемма (CR_3 — Closure under neutral terms)

Если M — нейтральный (не абстракция) и $\forall M'. M \rightarrow_\beta M' \Rightarrow M' \in RED_\sigma$, то $M \in RED_\sigma$.

Доказательство: индукция по типу σ (сложная, но технически прямолинейная).

Ключевое следствие: RED_σ замкнут относительно обратных β -шагов (CR_2)

Fundamental Lemma (основная лемма)

Лемма (Fundamental Lemma)

Пусть $\Gamma = x_1:\sigma_1, \dots, x_n:\sigma_n \vdash M : \tau$.

Если $N_i \in \text{RED}_{\sigma_i}$ для всех i , то

$$M[x_1 := N_1, \dots, x_n := N_n] \in \text{RED}_{\tau}$$

Идея доказательства.

Индукция по выводу типизации $\Gamma \vdash M : \tau$:

- Var: $M \equiv x_i$ — тривиально, $N_i \in \text{RED}_{\sigma_i}$
- App: используем определение $\text{RED}_{\sigma \rightarrow \tau}$ и И.П.
- Abs: $M \equiv \lambda x. P$ — нужно показать $\forall N \in \text{RED}_{\sigma}. (\lambda x. P[\dots])N \in \text{RED}_{\tau}$
Используем И.П. для P и CR_2 (обратный β -шаг)



Теорема о сильной нормализации

Теорема (Strong Normalization для STLC)

Если $\vdash M : \sigma$ (замкнутый типизуемый терм), то $M \in \text{SN}$.

Доказательство.

- 1 По Fundamental Lemma: $M \in \text{RED}_\sigma$ (подстановка пуста)
- 2 По CR_1 : $M \in \text{RED}_\sigma \Rightarrow M \in \text{SN}$
- 3 Следовательно, $M \in \text{SN}$



Следствие (Консистентность логики)

Тип $\perp$ (пустой тип) необитаем: не существует замкнутого терма M такого, что $\vdash M : \perp$.

Вывод: типизация гарантирует терминацию вычислений; интуиционистская логика непротиворечива

Связь с семантикой Крипке

Удивительная связь: три подхода к семантике соответствия Карри-Ховарда сходятся!

Модели Крипке

Миры = состояния знания

Forcing = истинность

Категорная семантика

Presheaf categories = функторы

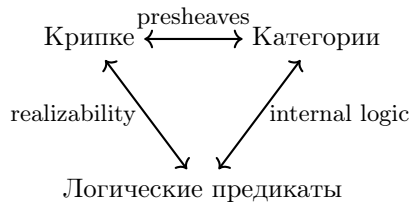
$W^{op} \rightarrow \mathbf{Set}$

Естественные преобразования

Нормализация

Reducibility = вычислимость

Индукция по типам



Unified view: все три подхода описывают одну математическую структуру с разных сторон

Обобщения и расширения

Метод логических предикатов обобщается на:

- ❶ System F (полиморфизм второго порядка)
 - Для $\forall$ используется метод кандидатов редуцируемости à la Girard (candidates of reducibility), корректно обрабатывающий импредикативность
 - Reducibility для $\forall\alpha.\sigma$: подстановка типов
- ❷ Зависимые типы (Martin-Löf Type Theory)
 - Нормализация $\equiv$ хорошая типизация
 - Logical predicates для Π -типов и Σ -типов
- ❸ Современные методы
 - Step-indexed logical relations — для языков с эффектами и рекурсией
 - Realizability — связь с теорией вычислимости
 - Semantic typing — refinement types (Liquid Haskell, F*)

Практическое применение: метод логических предикатов — основа верификации в Coq/Agda/Lean

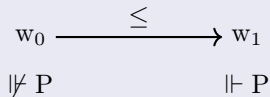
Двойное отрицание и модели Крипке

Вопрос: как модели Крипке соотносятся с вычислительным содержанием $\neg\neg A$?

Двойное отрицание в Крипке-модели

Формула $\neg\neg A$ может быть истинна, даже если A ложна:

Пример: линейная модель $w_0 \leq w_1$, где $w_0 \not\models P$, $w_1 \models P$



- $w_0 \models \neg\neg P$ (нет будущего мира, где P всегда ложно)
- Но $w_0 \not\models P$

Вывод: $\neg\neg$ — не «стирание доказательства», а слабая форма утверждения, модальность

Proof irrelevance в proof assistants

Различия между Coq и Lean

- Lean: пропозиции (Prop) proof-irrelevant по построению — любые два доказательства пропозиции считаются дефиниционально равными
- Coq: в Prop доказательства стираются при extraction (вычислительно не влияют), но proof irrelevance не доказывается без аксиомы Proof_Irrelevance — разные доказательства не обязательно равны. SProp (strict Prop) даёт proof-irrelevance без аксиом.
- В интуиционистской логике: $\neg\neg A \neq A$, но $\neg\neg A$ — «modalized» версия
- В категорной семантике: $\neg\neg$ — модальность (closure operator)

Ключевое различие

Proof irrelevance — свойство типовой системы (стирание/неразличимость доказательств)

Интуиционизм — свойство логики (конструктивность)

Эти концепции ортогональны и могут комбинироваться независимо

От моделей к верификации

Практические применения семантических методов:

- ❶ Kripke semantics $\rightarrow$ Modal logic $\rightarrow$ Temporal properties
 - LTL/CTL model checking — миры = состояния системы
 - Верификация протоколов, concurrent systems
- ❷ Логические предикаты $\rightarrow$ Refinement types
 - Liquid Haskell: $\{v:\text{Int} \mid v > 0\}$ — предикат в типе
 - F*: SMT-решатели + логические предикаты
- ❸ Normalization $\rightarrow$ Proof assistants
 - Coq/Agda/Lean: нормализация как вычисление доказательств
 - Extraction: доказательство $\rightarrow$ программа (гарантированно терминирующая)
- ❹ Категорная семантика $\rightarrow$ Program synthesis
 - Поиск программ через поиск морфизмов в категории
 - Type-directed synthesis (Hoogse, Djinn)

План лекции

- 1 Введение в формальные логики
- 2 Натуральный вывод
- 3 Изоморфизм доказательств и программ
- 4 Детуры, нормализация и консистентность
- 5 Классическая логика (NK)
- 6 Модели интуиционистской логики и нормализация
 - Семантика Крипке для интуиционистской логики
 - Нормализация через логические предикаты
- 7 Заключение

Соответствие Карри-Ховарда

Доказательства = Программы | Формулы = Типы

Ключевые темы:

- 1 Натуральный вывод NJ (интуиционистская логика) и НК (классическая)
- 2 Изоморфизм: логические связки $\leftrightarrow$ типы данных
- 3 Нормализация доказательств и устранение детуров
- 4 Интерпретация БГК: конструктивная семантика
- 5 Модели Крипке и контрмодели для классических формул
- 6 Метод логических предикатов (Tait) для сильной нормализации
- 7 Консистентность логики и корректность типизации

Практика: Coq, Agda, Lean, Isabelle — системы доказательств на основе этих идей

- Morten Heine Sørensen, Paweł Urzyczyn. Lectures on the Curry-Howard Isomorphism
- Philip Wadler. Propositions as Types
- Jean-Yves Girard, Paul Taylor, Yves Lafont. Proofs and Types
- Benjamin Pierce. Types and Programming Languages, Chapter 9
- Saul Kripke. Semantical Analysis of Intuitionistic Logic
- G. E. Mints. A Short Introduction to Intuitionistic Logic
- W. W. Tait. Intensional Interpretations of Functionals of Finite Type I
- Samuel Mimram. Program = Proof, Chapters 4–5 (Kripke models, normalization)